COMPUTE CREDIT INDEX RESEARCH

IOSCO ALIGNMENT GOVERNANCE PACKAGE

Pipeline & Publishing Workstream

CRI-H100 • CRI-A100 • CRI-L40S • CRI-H200 • CRI-B200 • CRI-R

Document Type	External Disclosure — IOSCO Governance Package
Document Version	v1.0
Effective Date	March 2026
Methodology Version	CCIR Methodology v1.0
Governance Framework	CCIR Governance Framework v1.0
Pipeline Version	1.1.1 (Databricks / Azure Delta Lake)
Audience	Rating Agencies • Legal Counsel • Credit Counterparties
IOSCO Reference	IOSCO Principles for Financial Benchmarks (July 2013)
Repository	github.com/ccir-index/ccir-methodology
Contact	research@ccir.io

Purpose of this Document

This document constitutes CCIR's formal IOSCO alignment disclosure for the pipeline and publishing workstream underpinning all CRI indices. It comprises three integrated components: (1) a Principle-by-Principle IOSCO mapping linking each of the 19 Principles to specific implemented pipeline controls; (2) a Controls and Procedures Narrative describing how the four-stage automated pipeline operationalizes those controls; and (3) an Audit Trail and Data Lineage Attestation documenting the complete Bronze-to-Silver-to-Gold provenance chain. This document should be read alongside the CCIR Governance Framework v1.0 and Methodology v1.0.

1. Pipeline Architecture Overview

The CCIR production pipeline operates as a four-stage automated workstream running on Azure Databricks with Delta Lake storage (Azure Data Lake Storage Gen2, account: ccirstorage, container: ccir-delta). All pipeline stages are implemented as Databricks Jobs. The pipeline progresses from raw market data collection (Bronze) through normalization and quality control (Silver) to index computation and publication (Gold). No human discretion is applied in any routine stage.

1.1 Job Schedule

Databricks Job	Notebooks Executed	Cadence	Primary Output
ingest_spot_primary	collect_vastai collect_shadeform collect_runpod	Daily 09:00 CT (primary)	bronze/spot_rental_raw (Delta Lake)
ingest_spot_fallback	collect_vastai collect_shadeform collect_runpod	Daily 21:00 CT (fallback)	bronze/spot_rental_raw (Delta Lake)
ingest_hyperscaler_daily	collect_azure collect_aws collect_gcp	Daily 00:00 UTC	bronze/hyperscaler_rate_raw (Delta Lake)
silver_transform_daily	transform_silver	Daily 10:00 AM CST (16:00 UTC)	silver/spot_rental (Delta Lake) Outlier flags applied
gold_compute_daily	compute_cri_h100 → compute_cri_r → compute_cri_a100 → compute_cri_l40s → compute_cri_h200 → compute_cri_b200 → export_snapshots	Daily (post-Silver)	gold/ Delta tables + GitHub ccir-data CSVs

1.2 Data Source Registry

The pipeline collects from three categories of data sources. Spot rental sources feed the benchmark indices (CRI-H100 and companion series). Hyperscaler sources feed CRI-R (reference rate cards). All sources are publicly available APIs with no human submission mechanism.

Source ID	Provider	Protocol	GPU Models Collected	Bronze Table
VASTAI_SPOT	Vast.ai	REST API	H100 SXM, A100 SXM4, A100 PCIe, L40S, H200, H200 NVL, B200, H100 PCIe, Tesla V100, RTX 4090	spot_rental_raw
SHADEFORM_MKT	Shadeform	REST API	H100, H200, A100, L40S, B200 (30+ providers normalized)	spot_rental_raw
RUNPOD_SPOT	RunPod	GraphQL	H100, H200, A100, L40S, B200, V100, RTX 4090 (spot / secure / community price types)	spot_rental_raw
AZURE_RETAIL	Azure	REST API (no	H100, A100 VM SKUs	hyperscaler_rate_raw

Source ID	Provider	Protocol	GPU Models Collected	Bronze Table
		auth)	(Consumption + DevTest)	
AWS_BULK	AWS	REST API (no auth)	p4d (A100), p4de (A100), p5 (H100) instance families; 6 regions	hyperscaler_rate_raw
GCP_BILLING	GCP	REST API (key auth)	A100, H100 SKUs from Compute Engine billing catalog (~31K SKUs paginated)	hyperscaler_rate_raw

2. IOSCO Principle-by-Principle Mapping

The following table maps each of the 19 IOSCO Principles for Financial Benchmarks (July 2013) to the specific pipeline controls and governance provisions implemented in the CCIR production workstream. Where a principle is addressed by multiple controls, the most directly operative pipeline control is cited. Full governance cross-references are provided to the CCIR Governance Framework v1.0 and Methodology v1.0.

#	IOSCO Principle	Pipeline Controls & Implementation Evidence	Governance Ref.	Status
P1	Overall Responsibility of the Administrator	CCIR acts as sole Administrator. All six Databricks Jobs (ingest_spot_primary, ingest_spot_fallback, ingest_hyperscaler_daily, silver_transform_daily, gold_compute_daily) are defined, scheduled, and owned by CCIR. Notebook names, source IDs, and methodology_version constants are set by CCIR. No third party has authority to modify pipeline logic.	Gov §2.1	**Full**
P2	Oversight of Third Parties to Whom Functions Are Delegated	No index calculation function is delegated. Vast.ai, Shadeform, RunPod, Azure, AWS, and GCP are treated as data vendors, not delegates. Source IDs are registered in the source registry and reviewed quarterly per Governance §9.3. Shadeform normalizes 30+ sub-providers in a single API response; CCIR performs its own canonical model mapping in transform_silver, ensuring CCIR retains calculation control regardless of upstream provider structure.	Gov §9.3	**Full**
P3	Conflicts of Interest for the Administrator	No submissions are accepted from any market participant. Raw data is sourced exclusively from public APIs (Vast.ai /bundles/, Shadeform /v1/instances/types, RunPod GraphQL, Azure Retail Prices, AWS per-region offer files, GCP billing catalog). collection_method field in Bronze records 'api_rest' or 'api_graphql' for every row, eliminating any human submission pathway. CCIR has no financial interest in any data source provider.	Gov §3; §2.4	**Full**
P4	Control Framework	Each collector notebook implements idempotent writes: before appending new data, a DELETE statement removes any existing rows for the same source_id and DATE(collected_at). This prevents duplicate accumulation on re-runs. Retry logic (3 attempts, 10-second backoff) is implemented in collect_vastai. All writes use Delta Lake ACID transactions. transform_silver deletes Silver rows over the full 8-day window before re-writing, ensuring Silver is always consistent with Bronze.	Gov §§2, 8, 9.2	**Full**
P5	Internal Oversight Function	Confidence flags (high/medium/low) are computed in each gold notebook and written to the Gold Delta table. The export_snapshots notebook publishes	Gov §§4.2, 4.3, 11.3	**Full**

#	IOSCO Principle	Pipeline Controls & Implementation Evidence	Governance Ref.	Status
		confidence_flag alongside every index value in the public CSV. n_valid_days and total_observations are also published, enabling downstream users to independently assess data quality. The Annual Benchmark Review (Governance §4.3) provides structured oversight of pipeline adequacy.		
P6	Benchmark Design	CRI-H100 uses a 7-day trailing window (window_start = now - 7 days, window_end = now), median of non-outlier spot offers (percentile_approx at 0.5), filtered to NVIDIA H100 SXM5 80GB, interconnect=SXM, price_type=spot, is_outlier=False. The 7-day window and median design are documented in Methodology §§5-6. WINDOW_DAYS = 7 is a named constant in compute_cri_h100, not a hard-coded literal.	Methodology §§5–6	**Full**
P7	Data Sufficiency	Confidence flags enforce minimum data requirements: high (>=5 valid days in window), medium (>=2 valid days), low (<2 valid days). A Valid Day requires observations from at least one collection run. If total_observations == 0, confidence_flag is forced to 'low' and cri_value is set to None rather than publishing a spurious value. Low-confidence values are published with disclosure, not suppressed.	Gov §4.3; Methodology §7.2	**Full**
P8	Hierarchy of Data Inputs	The pipeline enforces a strict three-tier hierarchy: (1) automated API collection per Methodology (no discretion); (2) Manifest Error Override by Administrator under §8.2 (documented and disclosed); (3) Fallback provisions per §7 (triggered by data unavailability, not discretion). The pipeline does not accept supplementary data or expert judgment in routine operation. RunPod's absence of geographic data is addressed by a documented source-aware override in transform_silver (region_raw='runpod_global' mapped to 'us'), reviewed quarterly.	Gov §2.4	**Full**
P9	Transparency of Benchmark Determinations	Every Gold row includes: observation_date, methodology_version, window_start, window_end, computed_at, confidence_flag, total_observations, n_valid_days, obs_p25, obs_p75, obs_min, obs_max. All fields are published in the public CSVs at github.com/ccir-index/ccir-data. export_snapshots publishes both daily snapshots and full history files. Commit messages include the observation date (e.g., 'Update CRI-H100 history through 2026-03-08'), creating a dated public audit trail.	Gov §§6, 8, 9.2	**Full**

#	IOSCO Principle	Pipeline Controls & Implementation Evidence	Governance Ref.	Status
P10	Periodic Review	The Annual Benchmark Review (Governance §4.3) covers: (i) relevance of Vast.ai as primary source; (ii) adequacy of secondary sources (Shadeform, RunPod); (iii) appropriateness of the 7-day window and median statistic; (iv) confidence threshold calibration; (v) source-aware region mapping (quarterly). RunPod's region override ('us' mapping) is explicitly designated for quarterly review in the transform_silver changelog.	Gov §4.3	**Full**
P11	Content of the Methodology	The complete methodology is published at github.com/ccir-index/ccir-methodology. methodology_version='1.1.1' (pipeline) / '1.2.0' (published methodology) is a named constant set at the top of every notebook and written to every Bronze and Gold row. The Methodology documents: data sources, quality filters, outlier removal (IQR x3), calculation window, confidence flags, restatement procedures, and fallback waterfall.	Methodology v1.0	**Full**
P12	Changes to the Methodology	Material changes require 60 days' notice; non-material changes require 14 days. METHODOLOGY_VER constant must be updated in every notebook for any version bump. Version history is maintained in Methodology v1.0 and the pipeline CHANGELOG.	Gov §5; Methodology Version History	**Full**
P13	Transition	Fallback waterfall defined in Governance §7: (1) prior published value carries for up to 5 Business Days; (2) confidence flag escalates to 'low'; (3) Administrator notifies known users within 2 Business Days. Benchmark replacement waterfall in §7.2 applies if no value published within 30 days. Fallback behavior is reflected in the pipeline: if total_observations == 0, cri_value is set to None and confidence_flag is 'low', signaling fallback to downstream consumers.	Gov §§7, 10.3–10.4	**Full**
P14	Submitter Code of Conduct	Not applicable. The CCIR pipeline accepts no submissions from market participants. All data is sourced via automated API collection. The 'no submissions' design is the core structural feature distinguishing CCIR from submission-based benchmarks (e.g., LIBOR). collection_method field explicitly records 'api_rest' or 'api_graphql' on every Bronze row, providing an auditable record that no submission pathway exists.	Gov §2.4	**N/A**
P15	Internal Controls over Data Collection	Each collector notebook implements: (i) idempotent source-scoped deletes before append; (ii) raw_payload stored as complete JSON string for every Bronze row, enabling re-processing from source data; (iii) ACID Delta Lake transactions prevent	Gov §9.2	**Full**

#	IOSCO Principle	Pipeline Controls & Implementation Evidence	Governance Ref.	Status
		partial writes; (iv) row_id (UUID) generated per offer at ingest, ensuring no two rows share an identifier; (v) collected_at is a UTC timestamp generated at collection time, not at write time. Source-level API failures raise exceptions (resp.raise_for_status()), preventing silent data gaps.		
P16	Complaints Procedures	Formal complaints procedure defined in Governance §12. Written complaints addressed to research@ccir.io. Acknowledgement within 5 Business Days; substantive response within 30 Business Days. All complaints and resolutions are retained in the complaints register. Pipeline Restatements triggered by valid complaints follow the Restatement procedure in §6.	Gov §12	**Full**
P17	Audits	Annual independent governance audit committed within 12 months of first publication. Pre-transaction audit available on request (rating agency and credit document use cases). Delta Lake time-travel enables historical recreation of any Bronze, Silver, or Gold state for audit purposes. All computed_at timestamps are UTC, providing a precise audit timeline.	Gov §11	**Committed**
P18	Audit Trail	Full Bronze-to-Silver-to-Gold lineage is maintained in Delta Lake. Key chain: Bronze row_id → Silver bronze_row_id (foreign key) → Gold observation_date / computed_at. Bronze raw_payload stores the complete raw API response as a JSON string, enabling independent re-computation. export_snapshots commits methodology_version and computed_at to the public GitHub repository on every run, creating an immutable dated record of each publication event.	Gov §§10.1, 11.1	**Full**
P19	Cooperation with Regulatory Authorities	CCIR commits to cooperate with relevant regulatory authorities upon request, including providing access to pipeline logs, Delta Lake audit trails, raw_payload archives, and governance documentation. Contact: research@ccir.io. Formal commitment documented in Governance §13.	Gov §13	**Full**

3. Controls and Procedures Narrative

This section describes how the four pipeline stages operationalize IOSCO governance requirements. Each stage corresponds to a Databricks Job tier. Controls are described in terms of the specific code-level mechanisms that implement them, enabling independent technical verification against the published notebook source.

3.1 Stage 1 — Bronze: Data Collection

The Bronze stage collects raw market data from all registered sources and writes it to the bronze/spot_rental_raw and bronze/hyperscaler_rate_raw Delta Lake tables. Two Databricks Jobs operate at this stage: ingest_spot_primary (Vast.ai, Shadeform, RunPod) runs daily at 09:00 CT, with ingest_spot_fallback providing a secondary collection at 21:00 CT; ingest_hyperscaler_daily (Azure, AWS, GCP) runs once per day at 00:00 UTC.

3.1.1 Provenance Fields (All Bronze Tables)

Every Bronze row written by every collector notebook includes the following mandatory provenance fields. These fields are written at ingest time and are never modified by downstream stages.

Field	Type	Value and Governance Purpose
row_id	UUID (STRING)	Generated via uuid.uuid4() at ingest. Unique identifier for every raw listing. Serves as the primary key for Bronze-to-Silver lineage (referenced as bronze_row_id in Silver).
source_id	STRING	Registered source identifier: VASTAI_SPOT, SHADEFORM_MKT, RUNPOD_SPOT, AZURE_RETAIL, AWS_BULK, GCP_BILLING. Enables per-source audit and outlier computation in Silver.
collected_at	TIMESTAMP (UTC)	Set to datetime.now(timezone.utc) at the start of each collection run, before any API calls. All rows within a collection run share the same collected_at value, preserving the snapshot boundary.
collection_method	STRING	'api_rest' (Vast.ai, Shadeform, Azure, GCP) or 'api_graphql' (RunPod). Provides auditable evidence that no human submission pathway was used.
methodology_version	STRING	Named constant METHODOLOGY_VER = '1.1.1' set at the top of each notebook. Updated for every methodology version change. Enables reconstruction of which methodology rules applied to any historical data point.
ingest_pipeline	STRING	'databricks_job'. Identifies the pipeline infrastructure layer, distinguishing Databricks production runs from any ad-hoc notebook executions.
raw_payload	STRING (JSON)	Complete raw API response for the individual listing, stored as json.dumps(offer). Enables full re-processing from raw source data without requiring re-access to the source API. The primary mechanism for P18 (Audit Trail) compliance.

3.1.2 Idempotency Controls

Each collector notebook implements source-scoped idempotent writes. Before appending new data, the following DELETE pattern executes:

```
DELETE FROM delta.`{BRONZE_PATH}`
WHERE source_id = '{SOURCE_ID}'
AND DATE(collected_at) = '{today_str}'
```

This control prevents duplicate accumulation on pipeline re-runs, system retries, or manual re-execution. Delta Lake's ACID transaction semantics ensure the delete and subsequent append are atomic.

3.1.3 API Resilience Controls

The collect_vastai notebook implements per-model retry logic with exponential backoff: up to 3 attempts per GPU model, 10-second wait between retries. If all retries are exhausted, the failure is logged and collection continues for remaining models. All collector notebooks call resp.raise_for_status(), raising an exception on any non-2xx HTTP response and preventing silent data gaps from being written as empty records.

3.2 Stage 2 — Silver: Normalization and Quality Control

The silver_transform_daily job runs daily at 10:00 AM CST (16:00 UTC) and produces the silver/spot_rental Delta table. Silver is the quality-controlled input to all Gold computation notebooks. Two substantive controls are applied in this stage: canonical model normalization and IQR-based outlier flagging.

3.2.1 Canonical Model Normalization

Raw GPU model strings from heterogeneous sources (e.g., 'H100_nvl', 'H100 SXM', 'A100_80G_SXM4') are mapped to canonical names via a deterministic lookup dictionary (MODEL_LOOKUP). The lookup is contains-based and ordered, with more specific keys before general ones to prevent misclassification (e.g., 'B200 SXM' is checked before 'B200'). The canonical model names written to Silver are:

- NVIDIA H100 SXM5 80GB (interconnect: SXM)
- NVIDIA H100 PCIe 80GB (interconnect: PCIe)
- NVIDIA H100 NVL 94GB (interconnect: NVLink)
- NVIDIA H200 SXM5 141GB (interconnect: SXM)
- NVIDIA H200 NVL 141GB (interconnect: NVLink)
- NVIDIA A100 SXM4 80GB (interconnect: SXM)
- NVIDIA A100 PCIe 80GB (interconnect: PCIe)
- NVIDIA L40S 48GB (interconnect: PCIe)
- NVIDIA B200 SXM 192GB (interconnect: SXM)
- NVIDIA Tesla V100 32GB / NVIDIA RTX 4090 24GB (reference/prosumer tiers)

Rows that do not match any key are written with gpu_model_canonical = 'unknown' and filtered out of Silver. This ensures only explicitly classified hardware contributes to any index computation.

3.2.2 Region Normalization (Source-Aware)

Geographic metadata is normalized to standardized buckets (us, eu, asia, other) via a source-aware UDF. RunPod does not expose geographic data via its GraphQL API; region_raw is set to 'runpod_global' at Bronze ingest and mapped to 'us' in Silver based on RunPod's predominantly US-hosted secure cloud infrastructure. This override is explicitly documented in the transform_silver notebook changelog and is subject to quarterly review per Governance §9.3.

3.2.3 Outlier Flagging

Outlier detection uses an IQR x3 fence computed per source_id and gpu_model_canonical group. The procedure:

- Computes Q1 and Q3 (25th and 75th percentiles) of price_per_gpu_hr_usd_raw using percentile_approx per group
- Computes IQR = Q3 - Q1
- Sets lower_fence = Q1 - 3 x IQR and upper_fence = Q3 + 3 x IQR
- Flags is_outlier = True for any row outside the fence

Outlier rows are retained in Silver with is_outlier = True and are excluded from all Gold computations by the filter is_outlier == False. Retaining outlier rows preserves the full data record for audit purposes while preventing outlier contamination of published index values.

3.2.4 Silver Lineage Fields

Silver adds the following lineage fields to each row: silver_id (new UUID, generated via spark's uuid() function), and bronze_row_id (copied from Bronze row_id). The silver_id-to-bronze_row_id chain provides complete Bronze-to-Silver traceability for every observation that enters Gold computation.

3.3 Stage 3 — Gold: Index Computation

The gold_compute_daily job executes six computation notebooks in sequence (compute_cri_h100 → compute_cri_r → compute_cri_a100 → compute_cri_l40s → compute_cri_h200 → compute_cri_b200), followed by export_snapshots. Each index notebook is mechanically identical except for the hardware filter applied to Silver.

3.3.1 CRI-H100 Computation (Representative for All Indices)

The compute_cri_h100 notebook applies the following filter chain to Silver:

```
gpu_model_canonical == 'NVIDIA H100 SXM5 80GB'
interconnect == 'SXM'
price_type == 'spot'
is_outlier == False
collected_at >= window_start (7 days prior, UTC)
collected_at <= window_end (current UTC time)
```

The index value is computed as percentile_approx(price_per_gpu_hr_usd, 0.5) — the approximate median — across all qualifying observations in the 7-day window. Percentiles at P25 and P75, and Min/Max, are computed in the same aggregation pass and published alongside the index value.

3.3.2 Confidence Flag Computation

Valid days are counted by grouping the filtered dataset by obs_date = to_date(collected_at) and counting distinct dates. The confidence flag is then assigned:

```
high: n_valid_days >= 5
medium: n_valid_days >= 2
low: n_valid_days < 2
```

If total_observations == 0, confidence_flag is forced to 'low' and cri_value is set to None. A None value is written to the Gold table and published in the CSV, signaling to downstream users that no valid index value exists for that observation date and that fallback provisions apply.

3.3.3 Gold Write Controls

Each Gold notebook implements idempotent writes at the observation_date level: before appending the new Gold row, any existing row for the same observation_date is deleted from the Gold Delta table. This ensures that re-runs of the daily job produce a clean, non-duplicated Gold series.

3.4 Stage 4 — Gold: Export and Publication

The export_snapshots notebook commits all index series to the public ccir-index/ccir-data GitHub repository using the GitHub Contents API. This constitutes the formal publication event for each daily index value.

3.4.1 Publication Mechanism

For each index, export_snapshots performs a GitHub API upsert (PUT /repos/{repo}/contents/{path}). The upsert: (1) checks whether the file exists and retrieves its current SHA; (2) base64-encodes the CSV content; (3) writes the new content with the existing SHA if updating, or without SHA if creating. The commit message includes the observation date (e.g., 'Update CRI-H100 history through 2026-03-08'), creating a datestamped, immutable record in the public GitHub commit history.

3.4.2 Published Files

File Path (ccir-index/ccir-data)	Update Frequency	Content
cri-h100/CRI-H100-history.csv	Daily	Full history: observation_date, methodology_version, cri_h100, obs_p25, obs_p75, obs_min, obs_max, confidence_flag, total_observations, n_valid_days, window_start, window_end, computed_at
cri-r/CRI-R-{date}.csv	Daily	Daily snapshot: source_id, gpu_model_canonical, instance_tier, rate_median_usd, rate_min/max_usd, sku_count, methodology_version
cri-r/CRI-R-history.csv	Daily	Full CRI-R rate card history across all providers and GPU models
cri-a100/CRI-A100-history.csv	Daily	Full history: same schema as CRI-H100-history.csv with index_name field
cri-l40s/CRI-L40S-history.csv	Daily	Full CRI-L40S history
cri-h200/CRI-H200-history.csv	Daily	Full CRI-H200 history
cri-b200/CRI-B200-history.csv	Daily	Full CRI-B200 history

4. Audit Trail and Data Lineage Attestation

This section documents the complete Bronze-to-Silver-to-Gold provenance chain maintained by the CCIR production pipeline. Every published index value is traceable, without gap, to a specific raw API response stored in the Bronze Delta Lake archive. The lineage chain is enforced by primary/foreign key relationships carried through each pipeline stage.

4.1 Lineage Chain Architecture

The following diagram summarizes the key identifier chain from source API to published CSV:

Vast.ai API offer object (raw JSON) ↓ stored in Bronze.raw_payload ↓ Bronze.row_id (UUID, generated at ingest) ↓ Silver.bronze_row_id (foreign key = Bronze.row_id) ↓ Silver.silver_id (new UUID per Silver row) ↓ Gold.observation_date + computed_at (aggregated from Silver qualifying set) ↓ Published CSV: cri-h100/CRI-H100-history.csv (GitHub commit, dated)

At each stage, the complete raw state is recoverable: Bronze raw_payload enables re-derivation of price_per_gpu_hr_usd_raw from the original offer object. Silver retains the bronze_row_id for every row that contributed to any Gold computation. Gold records the window_start, window_end, and computed_at, defining the exact Silver population used for each daily publication.

4.2 Field-Level Provenance Reference

Layer	Field	Data Type	Lineage and Governance Purpose
BRONZE	row_id	UUID STRING	Generated at ingest via uuid.uuid4(). Primary key. Referenced as bronze_row_id in Silver. Enables one-to-many Bronze-to-Silver tracing (single Bronze row can contribute to multiple Silver runs).
BRONZE	source_id	STRING	Registered source identifier (e.g., VASTAI_SPOT). Enables per-source audit isolation. Used as grouping key in Silver IQR outlier computation.
BRONZE	collected_at	TIMESTAMP UTC	UTC timestamp set at collection run start. Shared by all rows in a collection run. Defines the snapshot boundary for Silver's 8-day window filter.
BRONZE	collection_method	STRING	api_rest or api_graphql. Provides auditable evidence of no human submission pathway. Required for P3 and P14 compliance attestation.
BRONZE	methodology_version	STRING	Version of CCIR Methodology in effect at time of collection (e.g., '1.1.1'). Written to every row. Enables version-conditional reprocessing of historical data.
BRONZE	raw_payload	STRING (JSON)	Complete raw API response for each offer, stored as json.dumps(). The ground-truth archive for re-computation. Enables independent verification without requiring re-access to source APIs. Core P18 mechanism.
SILVER	silver_id	UUID STRING	New UUID generated at Silver transform time. Primary key for Silver rows.
SILVER	bronze_row_id	UUID STRING	Foreign key = Bronze.row_id. Provides direct traceability from any Silver row back to its Bronze

Layer	Field	Data Type	Lineage and Governance Purpose
			source record and raw_payload.
SILVER	gpu_model_canonical	STRING	Canonical GPU name from MODEL_LOOKUP (e.g., 'NVIDIA H100 SXM5 80GB'). Used as the primary filter in all Gold computation notebooks.
SILVER	is_outlier	BOOLEAN	IQR x3 outlier flag. All Gold notebooks filter is_outlier == False. Outlier rows are retained in Silver for auditability.
SILVER	region_normalized	STRING	Source-aware normalized region (us/eu/asia/other). RunPod override documented in notebook changelog.
GOLD	observation_date	DATE	The calendar date of the publication event (= window_end.date()). Primary key for each index series row. Used as the idempotent delete key before each daily write.
GOLD	methodology_version	STRING	Carried from Silver (originally from Bronze METHODOLOGY_VER constant). Enables version-conditional interpretation of any historical Gold value.
GOLD	window_start / window_end	TIMESTAMP UTC	Exact boundaries of the 7-day calculation window. Defines the complete Silver row population used for this index computation. Enables independent recreation of any historical calculation.
GOLD	computed_at	TIMESTAMP UTC	UTC timestamp of the Gold computation run. Together with window_start/window_end, fully specifies the temporal context of every published value.
GOLD	confidence_flag	STRING	high/medium/low based on n_valid_days. Published in all CSV outputs. None value written if total_observations == 0.
GOLD	n_valid_days / total_observations	INTEGER	Published alongside every index value. Enables downstream users to independently assess data quality. Required for P7 and P9 compliance.
CSV	methodology_version	STRING	Included in every published CSV row. Enables users referencing CCIR indices in legal documents to verify that the version in effect on any historical date matches the pinned methodology version in their contract.
CSV	computed_at	TIMESTAMP UTC	Published in all history CSVs. Combined with GitHub commit date, provides dual-layer temporal evidence for each publication event.

4.3 Delta Lake Time-Travel Audit Capability

All Bronze, Silver, and Gold tables are stored in Delta Lake format on Azure Data Lake Storage Gen2. Delta Lake's versioned transaction log (stored at /_delta_log/) enables point-in-time table reconstruction for any historical timestamp. This provides the following audit capabilities:

- Reconstruction of the Bronze table state at any prior collected_at timestamp, enabling verification of the exact data that was available for any historical Silver transform run
- Reconstruction of the Silver table state at any prior window boundary, enabling verification of the exact observation set that produced any historical Gold value

- Full transaction history (insert, delete, append operations) with timestamps, user IDs, and operation metadata
- Cross-version comparison of any Gold series before and after a pipeline change

4.4 GitHub Commit History as External Audit Record

The ccir-index/ccir-data public GitHub repository provides an independent, externally verifiable audit record of every publication event. Each export_snapshots run creates a dated commit with a structured commit message. The GitHub commit history is immutable from CCIR's perspective (commits cannot be rewritten without public visibility via force-push detection). This creates a dual-layer audit trail: the internal Delta Lake transaction log, and the external public GitHub commit history, both timestamped and independently verifiable.

For any publication date, an auditor can verify: (1) the exact CSV content published (via GitHub blob at that commit SHA); (2) the timestamp of publication (via commit timestamp); (3) the methodology_version in effect (via the methodology_version column in the published CSV); and (4) whether any restatement was applied (via a subsequent commit to the same file for the same observation date).

4.5 Restatement Procedure and Audit Trail

In the event of a Manifest Error (as defined in Methodology v1.0), a Restatement is processed as follows in the pipeline:

- The affected Gold row is identified by observation_date
- The idempotent delete-before-append pattern is applied to remove the erroneous row
- The corrected row is computed and appended with the original observation_date and an updated computed_at timestamp
- export_snapshots re-runs, creating a new GitHub commit to the history file with a commit message identifying the restatement
- The original erroneous commit remains in GitHub commit history; the restatement is additive, not destructive

Restatements are disclosed on the CCIR Substack and at ccir.io, with the GitHub commit SHA of both the original publication and the restatement commit, per Governance §6.

5. Pipeline Controls Summary

The following table provides a consolidated reference of all pipeline controls implemented across the four-stage workstream, mapped to their implementing notebook, IOSCO principle, and control category.

ID	Control Description	Implementing Notebook(s)	IOSCO Principle(s)
C-01	UUID row_id generated per offer at Bronze ingest. Never modified downstream. Primary key for lineage chain.	All collect_* notebooks	P18 (Audit Trail), P15 (Internal Controls)
C-02	raw_payload stored as complete JSON string for every Bronze row.	All collect_* notebooks	P18 (Audit Trail), P15
C-03	methodology_version constant declared at notebook top; written to every Bronze and Gold row.	All notebooks	P11 (Methodology), P12 (Changes)
C-04	collected_at set to UTC timestamp at run start; shared across all rows in a collection run.	All collect_* notebooks	P18, P15, P9 (Transparency)
C-05	collection_method field records 'api_rest' or 'api_graphql' on every Bronze row.	All collect_* notebooks	P3 (Conflicts), P14 (No submissions)
C-06	Idempotent source-scoped DELETE before Bronze append (source_id + DATE(collected_at)).	All collect_* notebooks	P4 (Control Framework), P15
C-07	resp.raise_for_status() on all API calls; exceptions prevent silent empty-record writes.	All collect_* notebooks	P4, P7 (Data Sufficiency)
C-08	Retry logic: 3 attempts, 10-second backoff per GPU model.	collect_vastai	P4, P7, P13 (Transition)
C-09	IQR x3 outlier flagging per source_id and gpu_model_canonical group. Outlier rows retained in Silver.	transform_silver	P6 (Design), P7
C-10	Canonical model lookup: ordered, contains-based, deterministic. Unmatched rows excluded from Silver.	transform_silver	P6, P15, P8 (Data Hierarchy)
C-11	Source-aware region normalization. RunPod override documented in changelog; reviewed quarterly.	transform_silver	P8, P10 (Periodic Review)
C-12	Silver window delete (8 days) before append. Prevents duplicate accumulation on re-runs.	transform_silver	P4, P18
C-13	bronze_row_id foreign key written to every Silver row.	transform_silver	P18, P15
C-14	7-day trailing window (WINDOW_DAYS = 7 named constant) applied in all Gold notebooks.	All compute_cri_* notebooks	P6, P11
C-15	percentile_approx(0.5) as index value. Median is manipulation-resistant and non-discretionary.	All compute_cri_* notebooks	P6, P3 (Conflicts)

ID	Control Description	Implementing Notebook(s)	IOSCO Principle(s)
C-16	Confidence flag computed from n_valid_days: high (>=5), medium (>=2), low (<2).	All compute_cri_* notebooks	P5, P7, P9
C-17	If total_observations == 0: cri_value = None, confidence_flag = 'low'. No spurious value published.	All compute_cri_* notebooks	P7, P13
C-18	Gold idempotent DELETE on observation_date before append.	All compute_cri_* notebooks	P4, P18
C-19	window_start, window_end, computed_at written to every Gold row.	All compute_cri_* notebooks	P18, P9
C-20	GitHub upsert retrieves existing file SHA before write, ensuring atomic single-file updates.	export_snapshots	P4, P9
C-21	Commit messages include observation date. GitHub commit history is immutable external audit record.	export_snapshots	P18, P9
C-22	methodology_version published in every CSV row.	export_snapshots	P11, P12, P9
C-23	Delta Lake time-travel preserves full Bronze/Silver/Gold state for any historical timestamp.	Platform (Azure ADLS Gen2)	P17 (Audits), P18

5.1 Open Items and Forward Commitments

The following items represent governance commitments not yet fully operationalized in the current pipeline version, documented in accordance with IOSCO transparency requirements:

#	Item	Status	Target
1	Independent Oversight Board	Committed in Governance §4.2. Not yet constituted.	Within 12 months of first publication
2	Annual Independent Audit (P17)	Committed in Governance §11. First audit pending first full year of publication.	Within 12 months of first publication
3	CRI-S Residual Value Index	Planned under Methodology v2.0.0. Pending brokerage transaction data partnerships.	TBD — subject to data access
4	SHA-256 Snapshot Hashing (Methodology §4)	Implemented March 10, 2026. Methodology commitment fulfilled.	Complete — March 10, 2026

6. Attestation and Contact

This document constitutes CCIR's formal disclosure of the pipeline and publishing controls implemented in the CRI-H100, CRI-A100, CRI-L40S, CRI-H200, CRI-B200, and CRI-R production pipeline as of March 2026. The controls described herein are implemented in the production Databricks notebooks running on Azure Data Lake Storage Gen2 (account: ccirstorage, container: ccir-delta) and the ccir-index/ccir-data public GitHub repository. The information in this document is provided for informational purposes to support rating agency analysis, credit document due diligence, and governance review. CCIR makes commercially reasonable efforts to ensure accuracy but does not warrant that the pipeline is free from errors or interruptions. Users requiring additional technical documentation, pipeline access for independent verification, or pre-transaction audit services should contact CCIR at research@ccir.io.

Administrator: Compute Credit Index Research LLC (CCIR)

Contact: research@ccir.io

Website: ccir.io

Methodology Repository: github.com/ccir-index/ccir-methodology

Published Data Repository: github.com/ccir-index/ccir-data

Governance Framework: CCIR Governance Framework v1.0 (March 2026)

Methodology: CCIR Methodology v1.0 (March 2026)

Pipeline Version: Methodology version 1.1.1 (Databricks / Azure Delta Lake)

IOSCO Reference: IOSCO Principles for Financial Benchmarks (July 2013)

© 2026 Compute Credit Index Research LLC. Published under Creative Commons Attribution 4.0 International (CC BY 4.0). This document may be reproduced and cited with attribution.